

CS (Degree) — Overview, Strategy and Navigation

Elective 2: **Computer Science & Engineering (Degree)**, NPSC CTSE 2026.

This file is the map for the Degree elective. Read it once fully, then keep it open as your checklist.

COMPUTER SCIENCE (DEGREE) -- THE HEAVIEST OF YOUR THREE ELECTIVES

Paper-I Sat 10 October 2026 1:00 pm - 4:00 pm

Paper-II Mon 12 October 2026 9:00 am - 12:00 noon

Technical MCQ Mon 12 October 2026 1:00 pm - 3:00 pm

Today is 5 September 2026.

DAYS TO PAPER-I : 35

DAYS TO PAPER-II : 37 (and the MCQ the same afternoon)

BUT the CS (Diploma) papers land on 29-30 September. Your genuinely free, Degree-only window is 1-9 October -- NINE DAYS.

Everything before that is shared-core work that pays into both.

12 October is a DOUBLE DAY: 3-hour descriptive paper in the morning, then a 2-hour 100-question MCQ in the afternoon after a ~1 hour gap.

ONE MERCY: there is a rest day on 11 October between the two papers.

Plan to use it -- it is your only same-elective cram day. See section 7.

Table of contents

- [1. Exam structure and marking](#)
- [2. The strategic picture for the Degree elective](#)
- [3. Syllabus checklist — Paper-I](#)
- [4. Syllabus checklist — Paper-II](#)
- [5. What the 2025 paper actually asked](#)
- [6. Recommended study order](#)
- [7. Day-by-day plan, 1–12 October](#)
- [8. Exam-day tactics](#)
- [9. File index for this folder](#)

1. Exam structure and marking

Component	Questions	Marks	Duration	Date & time
Technical Descriptive Paper-I	Sections A/B/C	100	3 hrs	Sat 10 Oct, 1–4 pm
Technical Descriptive Paper-II	Sections A/B/C	100	3 hrs	Mon 12 Oct, 9 am–12 noon

Technical MCQ	100 Q × 2 marks	200	2 hrs	Mon 12 Oct, 1–3 pm
General English + GK (common paper)	—	100	3 hrs	(see admit card)
		500 per elective		

Descriptive paper internal structure (both Paper-I and Paper-II)

Section	Question numbers	Rule	Marks each	Total
A	Q1 – Q10	answer any 8 of 10	5	40
B	Q11 – Q15	answer any 3 of 5	10	30
C	Q16 – Q19	answer any 2 of 4	15	30
			Total	100

Confirmed against the 2025 CS (Degree) Paper-I booklet (2537_25 (I), Series A) and Paper-II. The structure is stable — plan around it.

Read the choice arithmetic carefully, because it changes what "prepared" means:

- Section A: 10 offered, 8 required → you may drop **2**.
- Section B: 5 offered, 3 required → you may drop **2**.
- Section C: 4 offered, 2 required → you may drop **2**.

So **6 of the 19 questions on each paper can be skipped entirely and you can still score 100/100**. You do not need every corner of the syllabus at 15-mark depth. What you need is:

1. **Breadth at 5-mark depth** across every unit — because Section A is drawn from *everywhere*, one or two questions per unit, and it is 40% of the paper.
2. **Four or five bankable deep areas** for Sections B and C — topics where you can write a 15-mark answer with a worked example and a diagram without hesitating.

MCQ marking — negative marking of one-third

Outcome	Score
Correct	+2
Wrong	$-1/3 \times 2 = -0.667$
Not attempted	0

Three wrong answers cancel one correct answer. The guessing arithmetic is worked out in [§8](#).

2. The strategic picture for the Degree elective

This is the one elective written at your own level. Unlike CS (Diploma), where the risk was unfamiliar computer-*literacy* content, here the risk is the opposite: the Degree paper asks genuinely technical questions that reward *depth and worked mechanism*, and it contains four blocks that are unique to this elective and cannot be borrowed from anywhere else.

Where the load actually sits

Shared with Diploma and/or Forensic (study once, use three times):

Digital logic	P-I	\$3	-> CORE_01
Comp. organisation	P-I	\$4	-> CORE_02
Data structures	P-I	\$2a	-> CORE_08
C / C++	P-I	\$1	-> CORE_09
Operating systems	P-II	\$1	-> CORE_03
Software engineering	P-II	\$2	-> CORE_07
DBMS + SQL	P-II	\$3	-> CORE_06
Networks	P-II	\$4	-> CORE_05
Web technologies	P-II	\$5	-> CORE_09 / CSD_0x

DEGREE-ONLY -- nothing else in your three electives touches these:

Algorithm analysis, design paradigms, NP-completeness	P-I	\$2b
System software and compilers	P-I	\$5
Mobile application development	P-I	\$6
Artificial Intelligence	P-II	\$6
Machine Learning	P-II	\$7

The consequence

The four degree-only blocks are roughly **30% of the Degree syllabus** but they account for **essentially 100% of the new learning** you have to do for this elective. Everything else is refresh-and-format-drill.

And they are disproportionately represented in the big-mark questions. In 2025:

- Paper-I Section C (15 marks) offered **compiler phases** and **divide-and-conquer vs greedy** — two of the four questions on offer were degree-only algorithm/compiler content.
- Paper-II Section C (15 marks) offered **building a knowledge base in AI**.
- Paper-II Section B (10 marks) offered **knowledge-level vs symbol-level learning** and **k-NN**.

Compilers and algorithm analysis are the two highest-yield things you can study for this elective.

They are **CSD_01** and **CSD_02** — the two largest files in this folder for exactly that reason.

Effort per unit

Compilers & system software	#####	(deep, worked examples)
Algorithm design & analysis	#####	(deep, worked examples)
AI / Machine Learning	#####	(learn the named topics)
Mobile app development	#####	(learn -- syllabus is specific)
Web tech: PHP	#####	(learn syntax properly)
OS / DBMS / Networks / SE	#####	(refresh + format drill)
Digital logic / COA	#####	(refresh -- already drilled for Diploma)
Data structures / C / C++	####	(refresh only)

3. Syllabus checklist — Paper-I

Tick a box only when you can (a) write a 5-mark answer from memory and (b) get 8/10 on a self-quiz of its facts.

Filename note: **CORE_xx_*.md** are the shared-core notes being written into **Materials_Shared**, and **CSD_03–CSD_05** are being written into this folder by a parallel process. The names below are the *conventional* ones. **Confirm the exact filenames by listing **Materials_Shared** and this folder before you go looking.**

Unit 1 — Basics and Programming

Ref	Topic	Where the notes are	Depth needed	Done
§1.1	Basic units of a computer system (von Neumann: input, memory, ALU, CU, output; system bus)	CORE_02_ComputerOrganisation.md	Hand-drawable block diagram	- []
§1.2	Algorithms — definition, properties (finiteness, definiteness, input, output, effectiveness)	CSD_01 §1 · CORE_08_DataStructures.md	5-marker; contrast with flowchart	- []
§1.3	Flowcharts — symbols, drawing, algorithm vs flowchart	CORE_09_Programming_C_CPP.md	Must be able to <i>draw</i> one	- []
§1.4	C/C++ data types, declarations, expressions, operator precedence, type conversion	CORE_09_Programming_C_CPP.md	Refresh; MCQ output-prediction drill	- []
§1.5	Functions — definition, call by value vs call by reference, recursion, scope	CORE_09_Programming_C_CPP.md	Be ready to write code	- []
§1.6	Pointers — declaration, initialisation, arithmetic, pointer/array duality, dangling & null	CORE_09_Programming_C_CPP.md	Asked in 2025 Q3. High confidence needed	- []
§1.7	Arrays — 1-D, 2-D, row/column major, address calculation, arrays as parameters	CORE_09_Programming_C_CPP.md · CORE_08	Address-calculation sums	- []

Unit 2 — Data Structure & Algorithm

Ref	Topic	Where the notes are	Depth needed	Done
§2.1	Primitive data types; abstract data types	CORE_08_DataStructures.md	Refresh	- []
§2.2	Arrays, stacks, queues (circular, deque, priority)	CORE_08_DataStructures.md	Asked 2025 Q4. Refresh + applications	- []
§2.3	Linked lists — singly, doubly, circular; vs array	CORE_08_DataStructures.md	Asked 2025 Q5. Refresh	- []
§2.4	Trees — binary, BST, traversals, AVL, heaps, B-tree / B+ tree	CORE_08_DataStructures.md	Refresh; traversal MCQs are guaranteed	- []
§2.5	Graphs — representation, BFS, DFS	CORE_08_DataStructures.md · CSD_01 §9	Refresh + complexity $O(V+E)$	- []
§2.6	Sorting — bubble, insertion, selection, merge, quick, heap; comparison table	CORE_08_DataStructures.md · analysis in CSD_01 §5	Asked 2025 Q12 (quick sort). Deep	- []
§2.7	Searching — linear, binary; interpolation	CORE_08_DataStructures.md · CSD_01 §5.2	Refresh + recurrence	- []
§2.8	Hashing — hash functions, collision resolution, load factor	CORE_08_DataStructures.md	Refresh	- []

§2.9	Symbol tables — organisation, operations, implementations	CSD_01 §10	Degree-only depth	- []
§2.10	Analysing algorithms; asymptotic notation (O , Ω , Θ , o , ω)	CSD_01 §2	Asked 2025 Q6. Formal definitions cold	- []
§2.11	Time complexity of loops and recursion; recurrence relations; Master Theorem	CSD_01 §3–4	Work the examples. 10/15-mark territory	- []
§2.12	Divide and conquer — binary search, merge sort, quick sort, Strassen	CSD_01 §5	Asked 2025 Q17 (15 marks). Deep	- []
§2.13	Greedy method — fractional knapsack, activity selection, Huffman, Prim, Kruskal, Dijkstra	CSD_01 §6	Asked 2025 Q17 (15 marks). Deep + worked Huffman	- []
§2.14	Dynamic programming — 0/1 knapsack, LCS, matrix chain, Floyd–Warshall, optimal BST	CSD_01 §7	Work the tables. Very likely 15-marker	- []
§2.15	Backtracking and branch & bound — n-queens, graph colouring, Hamiltonian cycle, TSP	CSD_01 §8	Medium — one solid 10-marker	- []
§2.16	Search and traversal techniques — BFS/DFS as search strategies, applications	CSD_01 §9	Medium	- []
§2.17	NP-completeness — P, NP, NP-hard, NP-complete, reduction, Cook's theorem	CSD_01 §11	MCQ-guaranteed (2025 had 2). Explain clearly	- []

Unit 3 — Logic Design

Ref	Topic	Where the notes are	Depth needed	Done
§3.1	Number systems and conversions	CORE_01_DigitalLogic.md	Already drilled for Diploma — must be fast	- []
§3.2	Binary arithmetic; 1's and 2's complement; signed representation; floating point	CORE_01_DigitalLogic.md	MCQ-guaranteed. Both directions	- []
§3.3	Boolean algebra, laws, De Morgan, logic functions, SOP/POS, canonical forms	CORE_01_DigitalLogic.md	Asked 2025 Q7. Refresh	- []
§3.4	Minimisation — K-map (2/3/4 variable), don't-cares, Quine–McCluskey	CORE_01_DigitalLogic.md	Drill until mechanical	- []
§3.5	Flip-flops — SR, JK, D, T; excitation tables; master–slave; race-around	CORE_01_DigitalLogic.md	Asked 2025 Q16 (15 marks). Deep, with conversions	- []
§3.6	Combinational circuit design; adders (half, full, ripple carry, carry look-ahead)	CORE_01_DigitalLogic.md	Asked 2025 Q14 (10 marks). Draw the diagram	- []

§3.7	Sequential circuit design; state diagrams and tables	CORE_01_DigitalLogic.md	Medium	- []
§3.8	Registers and counters — shift registers, ripple/synchronous, ring, Johnson, mod-N	CORE_01_DigitalLogic.md	MCQ-heavy. Memorise state counts	- []
§3.9	Decoders, encoders, multiplexers, demultiplexers, code converters (binary ↔ Gray, BCD)	CORE_01_DigitalLogic.md	MCQ-heavy. Know which is which cold	- []

Unit 4 — Computer Organization & Architecture

Ref	Topic	Where the notes are	Depth needed	Done
§4.1	Hardware/software organisation of a computer; stored-program concept; buses	CORE_02_ComputerOrganisation.md	Refresh	- []
§4.2	Instruction formats, instruction cycle, sequencing, register transfers	CORE_02_ComputerOrganisation.md	Refresh	- []
§4.3	Addressing modes — immediate, direct, indirect, register, register-indirect, indexed, base+offset, relative, autoincrement	CORE_02_ComputerOrganisation.md	Asked 2025 Q9 + MCQ Q44. Memorise with examples	- []
§4.4	CPU design; hardwired vs microprogrammed control; RISC vs CISC; pipelining and hazards	CORE_02_ComputerOrganisation.md	MCQ Q43, Q45. Refresh	- []
§4.5	Memory organisation — hierarchy, cache (direct/associative/set-associative), hit rate, interleaving, virtual memory	CORE_02_ComputerOrganisation.md	Asked 2025 Q13 (10 marks) + MCQ Q46. Deep	- []
§4.6	I/O organisation — memory-mapped vs isolated I/O, programmed I/O, interrupts	CORE_02_ComputerOrganisation.md	MCQ Q47. Refresh	- []
§4.7	DMA — controller, cycle stealing vs burst mode, DMA vs interrupt vs programmed I/O	CORE_02_ComputerOrganisation.md	Named explicitly in syllabus — high probability	- []

Unit 5 — System Software's & Compilers (*entirely degree-only*)

Ref	Topic	Where the notes are	Depth needed	Done
§5.1	System vs application software; the software hierarchy	CSD_02 §1	5-marker	- []
§5.2	Assemblers — one-pass vs two-pass; MOT, POT, ST, LT, base table; worked two-pass example	CSD_02 §2	10/15-mark territory	- []
§5.3	Macros and macro processors — definition, expansion, nested, conditional	CSD_02 §3	Named in syllabus. 5/10-marker	- []

§5.4	Loaders — compile-and-go, absolute, relocating, direct-linking, dynamic, bootstrap	CSD_02 §4	MCQ Q48 in 2025. Learn the five types cold	- []
§5.5	Linkers — static vs dynamic, relocation, symbol resolution, linker vs loader	CSD_02 §5	Asked 2025 Q10 (5 marks)	- []
§5.6	Editors — line, stream, screen, structure editors	CSD_02 §6	Light — 5-marker	- []
§5.7	Phases of a compiler — all six + symbol table + error handler, traced on one statement	CSD_02 §7	Asked 2025 Q18 (15 marks). BANKABLE — learn cold	- []
§5.8	Lexical analysis — tokens/lexemes/patterns, regex, FA, NFA→DFA, lex	CSD_02 §8	Deep — 10-marker + MCQs	- []
§5.9	Syntax analysis: grammars — CFG, derivations, parse trees, ambiguity, left recursion, left factoring	CSD_02 §9	Deep	- []
§5.10	Top-down parsing — recursive descent, LL(1), FIRST/FOLLOW, parsing table	CSD_02 §10	Work the FIRST/FOLLOW example. 15-marker	- []
§5.11	Bottom-up parsing — shift-reduce, operator precedence, LR(0)/SLR/CLR/LALR, item sets	CSD_02 §11	MCQ Q49 in 2025. Work an item-set construction	- []
§5.12	Semantic analysis — SDD, attribute grammars, S- and L-attributed, type checking, symbol table	CSD_02 §12	Named in syllabus. 10-marker	- []
§5.13	Intermediate code — three-address code, quadruples, triples, indirect triples	CSD_02 §13	Work the conversions. Very examinable	- []
§5.14	Code optimisation — local/global, basic blocks, flow graphs, DAG, loop optimisation, peephole	CSD_02 §14	Named explicitly in syllabus. 15-marker	- []
§5.15	Code generation — target code, register allocation, instruction selection	CSD_02 §15	10-marker	- []
§5.16	Error detection and recovery at each phase	CSD_02 §16	Named explicitly in syllabus. 5/10-marker	- []

Unit 6 — Mobile Application & Services

Ref	Topic	Where the notes are	Depth needed	Done
§6.1	Factors in developing mobile applications	CSD_03/CSD_04*	5/10-marker	- []
§6.2	Frameworks and tools	CSD_03/CSD_04*	Asked 2025 Q15 (10 marks)	- []
§6.3	Text-to-speech techniques	CSD_03/CSD_04*	5-marker	- []
§6.4	Designing the right UI; multichannel and multimodal UIs	CSD_03/CSD_04*	Asked 2025 Q19 (15 marks)	- []
§6.5	Storing and retrieving data; Android storage	CSD_03/CSD_04*	10-marker	- []
§6.6	Synchronisation and replication of mobile data	CSD_03/CSD_04*	10-marker	- []
§6.7	Packaging and deploying	CSD_03/CSD_04*	5-marker	- []

* Being written by a parallel process — list the folder to confirm which file.

4. Syllabus checklist — Paper-II

Ref	Topic	Where the notes are	Depth needed	Done
§1.1	OS introduction, evolution, generations	CORE_03_OperatingSystems.md	Refresh	- []
§1.2	Components and services of an OS	CORE_03_OperatingSystems.md	Asked 2025 Q1. Refresh	- []
§1.3	Process management — PCB, states, threads, IPC, synchronisation, deadlock	CORE_03_OperatingSystems.md	Deep; deadlock is a 15-marker	- []
§1.4	Scheduling algorithms — FCFS, SJF, SRTF, Priority, Round Robin , multilevel queue	CORE_03_OperatingSystems.md	Asked 2025 Q11 (10 marks). Drill Gantt sums	- []
§1.5	Memory management — contiguous, paging , segmentation , virtual memory, page replacement, logical vs physical address	CORE_03_OperatingSystems.md	Asked 2025 Q2 AND Q16 (15 marks). Deep	- []
§1.6	I/O management, disk scheduling, RAID	CORE_03_OperatingSystems.md	Refresh	- []
§1.7	File system management — allocation methods, directory structures	CORE_03_OperatingSystems.md	Refresh	- []
§1.8	Security and protection — access matrix, domains, threats, authentication	CORE_03_OperatingSystems.md	Refresh	- []
§2.1	Concept of systems; system characteristics	CORE_07_SoftwareEngineering.md	Refresh	- []
§2.2	Software process models — waterfall, prototype, spiral, incremental, RAD, agile	CORE_07_SoftwareEngineering.md	Asked 2025 Q3 (SDLC phases). Deep	- []
§2.3	Software project planning and management; WBS ; risk management	CORE_07_SoftwareEngineering.md	Asked 2025 Q4 (WBS). Refresh	- []
§2.4	Cost estimation — LOC, FP, COCOMO (basic/intermediate/detailed), Delphi	CORE_07_SoftwareEngineering.md	Asked 2025 Q17 (15 marks). Learn COCOMO formulas	- []
§2.5	Scheduling — Gantt charts, CPM/PERT, milestones	CORE_07_SoftwareEngineering.md	Medium	- []
§2.6	Requirement analysis — SRS, elicitation, functional vs non-functional, DFD	CORE_07_SoftwareEngineering.md	Refresh	- []
§2.7	Testing strategies — unit, integration, system, acceptance; black vs white box; regression	CORE_07_SoftwareEngineering.md	Deep — reliable 10/15-marker	- []

§2.8	Quality assurance — SQA activities, ISO 9126, CMM levels, verification vs validation	CORE_07_SoftwareEngineering.md	Refresh; memorise CMM levels	- []
§3.1	Data abstraction (3-level architecture), data models, data independence	CORE_06_DBMS_SQL.md	Refresh; logical vs physical independence	- []
§3.2	DDL / DML / DCL / TCL	CORE_06_DBMS_SQL.md	Refresh	- []
§3.3	Attributes, keys (super, candidate, primary, foreign, composite), integrity constraints	CORE_06_DBMS_SQL.md	Asked 2025 Q6. Refresh	- []
§3.4	Query processing and optimisation — parsing, translation to relational algebra, cost estimation, execution plan	CORE_06_DBMS_SQL.md	Asked 2025 Q18 (15 marks). Deep	- []
§3.5	Structure of relational databases; relational algebra; RDBMS characteristics / Codd's rules	CORE_06_DBMS_SQL.md	Asked 2025 Q5. Refresh	- []
§3.6	SQL by example — SELECT, joins, aggregates, GROUP BY/ HAVING, subqueries, views	CORE_06_DBMS_SQL.md	Be ready to write queries	- []
§3.7	Normalisation — 1NF→BCNF, functional dependencies	CORE_06_DBMS_SQL.md	Deep — classic 15-marker	- []
§3.8	Distributed databases — fragmentation, replication, transparency, 2-phase commit	CORE_06_DBMS_SQL.md	Named in syllabus — do not skip	- []
§3.9	Security and integrity; transactions, ACID, concurrency control, recovery	CORE_06_DBMS_SQL.md	Refresh	- []
§3.10	Network model (and hierarchical model) vs relational	CORE_06_DBMS_SQL.md	Named in syllabus — easily forgotten. Learn it	- []
§4.1	Basic network concepts; LAN/ MAN/WAN	CORE_05_ComputerNetworks.md	Refresh	- []
§4.2	Topologies — bus, star, ring, mesh, tree, hybrid	CORE_05_ComputerNetworks.md	Asked 2025 Q12 (10 marks). Table + diagrams	- []
§4.3	Networking devices — hub, switch, bridge, router, gateway, repeater, NIC	CORE_05_ComputerNetworks.md	Refresh; know the OSI layer of each	- []
§4.4	Transmission media — twisted pair, coax, fibre, wireless	CORE_05_ComputerNetworks.md	Refresh	- []
§4.5	OSI model — 7 layers, functions, PDUs, protocols per layer	CORE_05_ComputerNetworks.md	Asked 2025 Q7. Must be automatic	- []

§4.6	TCP/IP fundamentals — 4/5 layers, TCP vs UDP , three-way handshake, ports	CORE_05_ComputerNetworks.md	Asked 2025 Q8. Deep	- []
§4.7	Internet — DNS, HTTP, FTP, SMTP, DHCP, NAT	CORE_05_ComputerNetworks.md	Refresh	- []
§4.8	Internet routing — distance vector, link state, RIP/OSPF/BGP	CORE_05_ComputerNetworks.md	Named in syllabus. 10-marker	- []
§4.9	IP addressing — classes, subnetting, CIDR, IPv4 vs IPv6, private ranges	CORE_05_ComputerNetworks.md	Drill subnetting sums — MCQ gold	- []
§5.1	Website development phases — planning, design, implementation, testing, maintenance	CSD_0x (web file)*	Asked 2025 Q13 (10 marks)	- []
§5.2	HTML — structure (HTML/HEAD/TITLE/BODY), paragraphs, lists, formatted/unformatted text, hyperlinks, font, image	CSD_0x*	Learn the tags and attributes literally	- []
§5.3	HTML forms; GET vs POST	CSD_0x*	Asked 2025 Q9	- []
§5.4	PHP — server-side scripting, installing, embedding in HTML, syntax and variables	CSD_0x*	Asked 2025 Q10. Learn syntax properly	- []
§5.5	PHP — passing information between pages (GET, POST, sessions, cookies)	CSD_0x*	10-marker	- []
§5.6	PHP — basic errors and problems	CSD_0x*	5-marker	- []
§6.1	AI: knowledge and reasoning; building a knowledge base	CSD_0x (AI file)*	Asked 2025 Q19 (15 marks)	- []
§6.2	Learning from observations — inductive learning, explanation-based learning	CSD_0x*	Named explicitly. 10-marker	- []
§6.3	Pattern recognition — structured description, symbolic description, object identification, speech recognition	CSD_0x*	Named explicitly	- []
§6.4	LISP and PROLOG — features, syntax flavour, comparison	CSD_0x*	Named explicitly. Very MCQ-able	- []
§7.1	Knowledge-level vs symbol-level learning	CSD_0x (ML file)*	Asked 2025 Q14 (10 marks)	- []
§7.2	Supervised learning — distance-based methods, k-Nearest Neighbours	CSD_0x*	Asked 2025 Q15 (10 marks)	- []
§7.3	Unsupervised learning — K-means , kernel K-means	CSD_0x*	Named explicitly. Work an example	- []

§7.4	Statistical learning theory; ensembles — boosting, bagging, random forests	CSD_0x*	Named explicitly. 10-marker	- []
-------------	--	---------	-----------------------------	-------

* Being written by a parallel process — list the folder to confirm which file.

5. What the 2025 paper actually asked

Derived from the three solved 2025 CS (Degree) papers in `C:\Users\hp\Desktop\prep\_raw\existing_txt\` (Paper-I booklet 2537_25 (I) Series A; Paper-II; MCQ booklet 2537_25 (A) Series A). **This is the single most useful calibration data you have — read this section twice.**

5.1 The headline finding: the paper walks the syllabus in order

Paper-I Section A was, question by question, a walk down the six syllabus units in the order they are printed:

Q	Question (2025 Paper-I, Section A, 5 marks each)	Syllabus unit
1	Basic functional units of a computer system; role of each	§1 Basics
2	Differentiate an algorithm and a flowchart	§1 Basics
3	Pointers in C/C++ — declaration and initialisation	§1 Programming
4	Key differences between a stack and a queue	§2 Data structures
5	Linked list — concept, advantages over an array	§2 Data structures
6	Define Big-Oh; what it represents in algorithm analysis	§2 Algorithms
7	Boolean algebra; basic operators and their functionality	§3 Logic design
8	Combinational vs sequential circuits, with one example each	§3 Logic design
9	Addressing modes; give one example	§4 COA
10	Role of a linker in the compilation process	§5 System software

Implication: if you can produce a competent 5-mark answer on the two or three headline topics of each syllabus unit, you have Section A's 40 marks nearly banked. Section A is *breadth*, not depth, and it is highly predictable. Note also that Mobile Applications (§6) did **not** appear in Section A — it appeared in Sections B and C instead.

5.2 Paper-I Sections B and C

Q	Marks	Question	Unit
11	10	Write a C/C++ function to reverse an array ; explain the logic with an example	§1
12	10	Working principle of Quick Sort ; average and worst-case complexity	§2
13	10	Levels of memory organisation ; purpose of the memory hierarchy	§4
14	10	Design a 2-bit binary adder ; draw the logic diagram and explain	§3
15	10	Frameworks and tools in mobile application development	§6
16	15	Design and working of a JK flip-flop ; using it to implement other flip-flops	§3
17	15	Compare divide and conquer vs greedy , with a concrete example of each	§2

18	15	Phases of a compiler , lexical analysis to code generation; task and output of each	\$5
19	15	Challenges in designing the right UI for mobile applications	\$6

Notice the pattern: **Sections B and C also walk the units, one question per unit.** Section B covered units 1, 2, 4, 3, 6. Section C covered units 3, 2, 5, 6. So each big-mark section offers you roughly one question per major unit — which means **your choice of two 15-markers is effectively a choice of which two units you want to be examined deeply on.**

That is a gift. Pick your two strongest units in advance and drive them to 15-mark depth.

5.3 Paper-II Sections A, B and C

Q	Marks	Question	Unit
1	5	Key components of an OS; function of each	\$1 OS
2	5	Logical vs physical addresses in memory management	\$1 OS
3	5	Phases of the SDLC; brief note on each	\$2 SE
4	5	Purpose of a Work Breakdown Structure	\$2 SE
5	5	Key characteristics of an RDBMS	\$3 DBMS
6	5	Primary key and foreign key in database design	\$3 DBMS
7	5	OSI model; list the seven layers	\$4 Networks
8	5	TCP vs UDP	\$4 Networks
9	5	GET vs POST in HTML forms	\$5 Web
10	5	PHP syntax rules and use of variables	\$5 Web
11	10	Round Robin scheduling with an example; advantages and disadvantages	\$1
12	10	Network topologies — features, advantages, disadvantages	\$4
13	10	Phases of website development	\$5
14	10	Knowledge-level vs symbol-level learning	\$7 ML
15	10	k-Nearest Neighbours ; factors influencing performance	\$7 ML
16	15	Memory management techniques , including paging and segmentation	\$1
17	15	Cost estimation methods ; which is most reliable and why	\$2
18	15	Query processing steps in an RDBMS; why optimisation matters	\$3
19	15	Building a knowledge base in AI; key challenges	\$6

Paper-II Section A is exactly two questions per unit for units 1–5. AI and ML appeared only in B and C. That is a very clean pattern and a very safe planning assumption: expect **2 × 5-mark questions each on OS, SE, DBMS, Networks, Web**, and expect **AI and ML in the 10/15-mark sections.**

5.4 Difficulty and style calibration

Observation	What to do about it

Questions are textbook-standard. No trick questions, no unseen problem-solving, nothing research-flavoured. Every one is a question you would find at the end of a chapter.	Prepare <i>standard answers</i> , not clever ones. The examiner wants the canonical treatment.
Phrasing is short and direct. "Define X." "Differentiate A and B." "Explain the working of X with an example." "Discuss the challenges in Y."	Practise recognising the verb: <i>define/differentiate/explain/design/discuss/describe and compare</i> . Each demands a different shape of answer.
"Differentiate" and "compare" appear constantly — algorithm vs flowchart, stack vs queue, combinational vs sequential, TCP vs UDP, GET vs POST, logical vs physical address, knowledge-level vs symbol-level, divide-and-conquer vs greedy.	Build a comparison-table bank. A two-column table with 5–6 rows of contrast, plus a one-line "common ground" note, is a full-marks 5-mark answer and takes 6 minutes.
"Give one example" / "with an example" is attached to most 10-markers.	Never write a 10-mark answer without a worked example. Have a rehearsed example ready for every major algorithm.
Diagrams are expected — logic diagrams, flip-flop design, compiler phase diagram, topology sketches, memory hierarchy pyramid, Gantt charts.	Practise hand-drawing them to a 3-minute budget. A labelled diagram earns marks that prose does not.
15-markers ask for full treatment: derivation or diagram, several sub-parts, plus a judgement ("which is most reliable and why", "key challenges involved").	End every 15-mark answer with an evaluative paragraph. Examiners reward the answer that takes a position and justifies it.
Answer-length guidance from the solved set: 5 marks ≈ 5–6 solid points; 10 marks ≈ a worked explanation; 15 marks ≈ full treatment with derivation or diagram.	Budget: 5-mark = 8 min, 10-mark = 16 min, 15-mark = 24 min. That is 40 + 48 + 48 = 136 min of the 180, leaving ~25 min reading/choosing and ~20 min buffer.

5.5 What the MCQ paper looked like

The 2025 MCQ paper's own stated topic spread:

Question range	Topic block	Count	Share
Q1–30	Programming, data structures, algorithm analysis	30	30%
Q31–50	Digital logic, computer architecture, system software	20	20%
Q51–67	Operating systems, software engineering	17	17%
Q68–76	Databases	9	9%
Q77–89	Networks and web technology	13	13%
Q90–100	Artificial intelligence and machine learning	11	11%

Read that table as a study-hours allocation. Nearly a third of the MCQ is programming/DS/algorithms; a fifth is logic + architecture + system software. Together that is **half the MCQ paper and it maps almost exactly onto Paper-I**, which is why Paper-I preparation double-counts.

Representative MCQ items, to calibrate difficulty:

- Which component mitigates the Von Neumann bottleneck? → cache memory hierarchy
- Worst-case complexity of linear search → $O(n)$
- Which register holds the address of the next instruction? → Program Counter
- Output of `printf("%d", 5/2)` in C → 2
- Complexity of BFS and DFS on an adjacency-list graph → $O(V+E)$

- *Traversal visiting Left → Right → Root → postorder*
- *What it means for a decision problem to be in NP → a certificate can be verified in polynomial time*
- *2's complement of 1101 0010 → 0010 1110*
- *Propagation delay of a 4-bit ripple-carry adder at 2 ns per full adder*
- *Which loader function resolves external references between object files?*
- *Which parsing technique uses a stack and a parsing table?*

These are **single-fact and one-step-calculation questions**. They are not hard; they are wide. The failure mode is not difficulty, it is *breadth gaps* and *careless arithmetic*.

Note also, honestly: the solved set flags that **four of the hundred questions were genuinely ambiguous as printed**. Expect one or two unanswerable-as-written items. Do not burn five minutes on them — mark and move.

5.6 The five things this analysis should change about your preparation

1. **Section A is predictable and cheap. Secure it first.** Two headline 5-markers per unit × 13 units ≈ 26 rehearsed short answers gets you most of 80 marks across both papers.
2. **Pick your Section C units now.** For Paper-I the natural picks are **compilers** (CSD_02 §7) and **algorithms** (CSD_01 §5–7); flip-flops is a strong third. For Paper-II: **memory management** and **cost estimation** or **query processing**.
3. **Build the comparison-table bank.** It is the single highest-yield revision artefact for this paper's style.
4. **Rehearse diagrams to a stopwatch.** Compiler phases, memory hierarchy, logic diagrams, topologies.
5. **The MCQ rewards breadth, not depth.** In the last week, switch from deep study to wide, fast fact-recall across all thirteen units.

6. Recommended study order

The constraint

Diploma Paper-I and Paper-II are on **29–30 September**. Everything from today until then is Diploma-priority — but the shared core you build for Diploma (digital logic, COA, OS, DBMS, networks, SE, C) is *exactly* the shared core the Degree papers need. So the plan is:

5 -- 28 Sept	Diploma-priority. Shared core built once, banked for both. Steal 30–45 min on 3–4 evenings a week for CSD_01 and CSD_02 ONLY -- the two degree-only blocks that need slow absorption.
29 -- 30 Sept	Diploma exams. Do not study Degree material.
1 -- 9 Oct	THE DEGREE WINDOW. Nine days. This is where the plan below lives.
10 Oct	Paper-I, 1–4 pm.
11 Oct	The gift day. Paper-II cram.
12 Oct	Paper-II 9–12, MCQ 1–3.

Why start compilers early rather than in the window

CSD_02 (system software and compilers) is the one block that **cannot be crammed**. FIRST/FOLLOW sets, LR item-set construction and three-address code are mechanical skills — they need repetition spread over days, not a single long sitting. Nobody learns to build an SLR parsing table the night before. Start it in September at low intensity even while Diploma has priority; by 1 October you want it to feel *familiar*, so that the October window is revision rather than first contact.

The same is true, to a lesser degree, of the DP tables in [CSD_01](#) §7. Do 0/1 knapsack and LCS by hand three times across September and they will be automatic.

Ordering principle for the nine-day window

1. **Degree-only material first** (compilers, algorithms, mobile, AI, ML) — it is new learning, and new learning needs the days *before* the exam, not the hours.
2. **Shared core second** — it is refresh, and refresh survives being done late.
3. **Paper-I material before Paper-II material**, because Paper-I is first and there is a rest day before Paper-II.
4. **MCQ breadth-drilling last**, in the final 48 hours, because it is recall and recall is freshest when recent.

7. Day-by-day plan, 1–12 October

Assume 3–4 focused hours per day, split morning/evening. Each day: **new material first, revision of yesterday second, 20 MCQs last**.

Date	Day	Morning block (~2 hrs)	Evening block (~1.5 hrs)	Target
Wed 1 Oct	Decompress + pivot	Read this file end to end. Re-read CSD_02 §7 (compiler phases) and write the full phase diagram from memory .	CSD_02 §8 lexical analysis; NFA→DFA once by hand.	Compiler phases 15-marker locked
Thu 2 Oct	Compilers I	CSD_02 §9–10: CFG, ambiguity, left recursion, left factoring, FIRST/FOLLOW + LL(1) table worked twice .	CSD_02 §2–3: assemblers, two-pass example, macros.	LL(1) table mechanical
Fri 3 Oct	Compilers II	CSD_02 §11: shift-reduce, LR(0) item sets and SLR table worked once fully .	CSD_02 §4–6: loaders, linkers, editors — pure memorisation, make a one-page table.	Loader types cold
Sat 4 Oct	Compilers III	CSD_02 §13–16: three-address code, quads/triples/indirect triples, optimisation, basic blocks, DAG, peephole , code generation, error recovery.	Re-do FIRST/FOLLOW and the item-set example from a blank page .	Unit 5 complete
Sun 5 Oct	Algorithms I	CSD_01 §2–5: asymptotics, loop and recursive analysis, recurrences and Master Theorem — work all cases , divide and conquer with recurrences.	CSD_01 §6: greedy; Huffman worked fully ; Prim/Kruskal/Dijkstra traced on one graph.	Master Theorem automatic
Mon 6 Oct	Algorithms II	CSD_01 §7: DP — 0/1 knapsack table and LCS table both worked by hand , matrix chain, Floyd–Warshall, optimal BST. DP vs D&C.	CSD_01 §8–11: backtracking, branch and bound, traversal, symbol tables, NP-completeness .	Unit 2 complete
Tue 7 Oct	Paper-I shared core	CORE_01 digital logic: K-maps, flip-flops (JK + conversions), adders, counters, decoders/MUX — drill, do not read.	CORE_02 COA: addressing modes, memory hierarchy, cache mapping, DMA.	Units 3+4 refreshed
Wed 8 Oct	Paper-I remainder	Mobile applications (CSD_0x) — all seven syllabus sub-topics, since 2025 gave it a 10 and a 15.	CORE_08 data structures + CORE_09 C/C++: write reverse-array, linked-list insert, BST traversal on paper .	Units 1+6 refreshed

4. **Start with Section C if your two 15-markers are strong.** 30 marks in the first 50 minutes while fresh is worth more than 30 marks in the last 50 while rushing. (If you are prone to over-writing, do Section A first instead so you cannot run out of time on cheap marks.)

Time budget

Section	Questions	Minutes each	Total
Reading and choosing	—	—	10
A	8 × 5	8	64
B	3 × 10	16	48
C	2 × 15	22	44
Review / diagrams tidy	—	—	14
			180

Hard rule: when a question's clock runs out, stop and move on, even mid-sentence. An unanswered 5-marker costs 5 marks; an over-written 15-marker costs you a whole other question.

Answer shape by mark value

Marks	Shape
5	One-line definition → 5–6 bullet points → one closing "key point for full marks" line. For a "differentiate" question: a table, then one line of common ground. Roughly half a page to three-quarters.
10	Intro paragraph → mechanism/steps → worked example or diagram → advantages/disadvantages table → one-line conclusion. Roughly 1.5 pages.
15	Intro → full structured treatment with numbered sub-headings → worked example AND a labelled diagram → comparison or evaluation → a judgement paragraph that answers the "why/which/what challenges" part explicitly. Roughly 2.5–3 pages.

Always underline or box key terms and headings. Examiners scan for keywords. A well-structured answer with headings scores above a better-written wall of text.

MCQ — the guessing arithmetic

With +2 correct and −0.667 wrong, over a long run of guesses:

Situation	Expected value	Verdict
Blind guess from 4 options	$(0.25 \times 2) + (0.75 \times -0.667) = \mathbf{-0.0}$	Break-even. Neutral.
One option eliminated (1 in 3)	$(0.333 \times 2) + (0.667 \times -0.667) = \mathbf{+0.22}$	Guess.
Two options eliminated (1 in 2)	$(0.5 \times 2) + (0.5 \times -0.667) = \mathbf{+0.67}$	Definitely guess.
Genuinely no idea, no elimination	≈ 0	Skip — it costs nothing and saves time.

Rule: if you can eliminate even one option, answer. If you cannot eliminate anything, leave it blank and spend the seconds elsewhere.

The break-even at 1-in-4 is exactly zero, which means blind guessing is not *punished* — but it is not rewarded either, and it wastes time you could spend on a question you can actually crack. Blank is the disciplined choice.

MCQ passes

- **Pass 1 (~60 min):** answer everything you know instantly. Mark anything needing calculation with a dot, anything you have no idea about with a cross. Do not linger — 100 questions in 120 minutes is 72 seconds each.
- **Pass 2 (~40 min):** the dotted ones. Do the arithmetic — complement conversions, propagation delays, complexity counts, subnet masks.
- **Pass 3 (~15 min):** the crossed ones. Eliminate what you can; answer if you get down to three or fewer; leave the rest.
- **Pass 4 (~5 min):** verify the answer sheet is correctly aligned with question numbers. A one-row shift is catastrophic and it does happen.

The double day, 12 October

You have roughly one hour between a 3-hour descriptive paper and a 2-hour MCQ. That gap is not study time, it is recovery time.

- Eat something light with protein, not heavy carbohydrate. Water, not a large coffee.
- Ten minutes of walking outside, away from other candidates and their post-mortems.
- Twenty minutes maximum of rapid fact-flipping from a **single pre-prepared one-page sheet** — complexities, OSI layers and port numbers, loader types, CMM levels, cache mappings, flip-flop excitation table. Prepare this sheet on 8 October, not on the day.
- **Do not** discuss Paper-II answers with anyone. It only creates doubt you cannot act on.
- Build the stamina for this in advance: on 9 October do the mock Paper-I in the morning and a 100-question MCQ in the afternoon, back to back.

9. File index for this folder

File	Covers	Status
CSD_00_OVERVIEW.md	This file — dates, structure, checklists, 2025 analysis, plan	✓
CSD_01_AlgorithmsAndAnalysis.md	Paper-I §2 degree-only depth: asymptotics, recurrences and Master Theorem, divide and conquer, greedy, dynamic programming, backtracking, branch and bound, traversal, symbol tables, NP-completeness	✓
CSD_02_SystemSoftwareAndCompilers.md	Paper-I §5 entire: assemblers, macros, loaders, linkers, editors; compiler phases, lexical, syntax (LL and LR), semantic, intermediate code, optimisation, code generation, error recovery	✓
CSD_03 / CSD_04 / CSD_05	Mobile applications, web technologies, AI and machine learning (being written by a parallel process — list this folder for the exact names)	—

Shared-core files (in `Materials\_Shared\`)

These are being written by other processes. **Do not duplicate them here.** Conventional names — verify by listing the folder:

File	Serves Degree units
OFFICIAL_SYLLABUS.md	Ground truth. Read first, believe nothing that contradicts it.